REPORTE EJECUTIVO DE CUMPLIMIENTO DE REQUERIMIENTOS

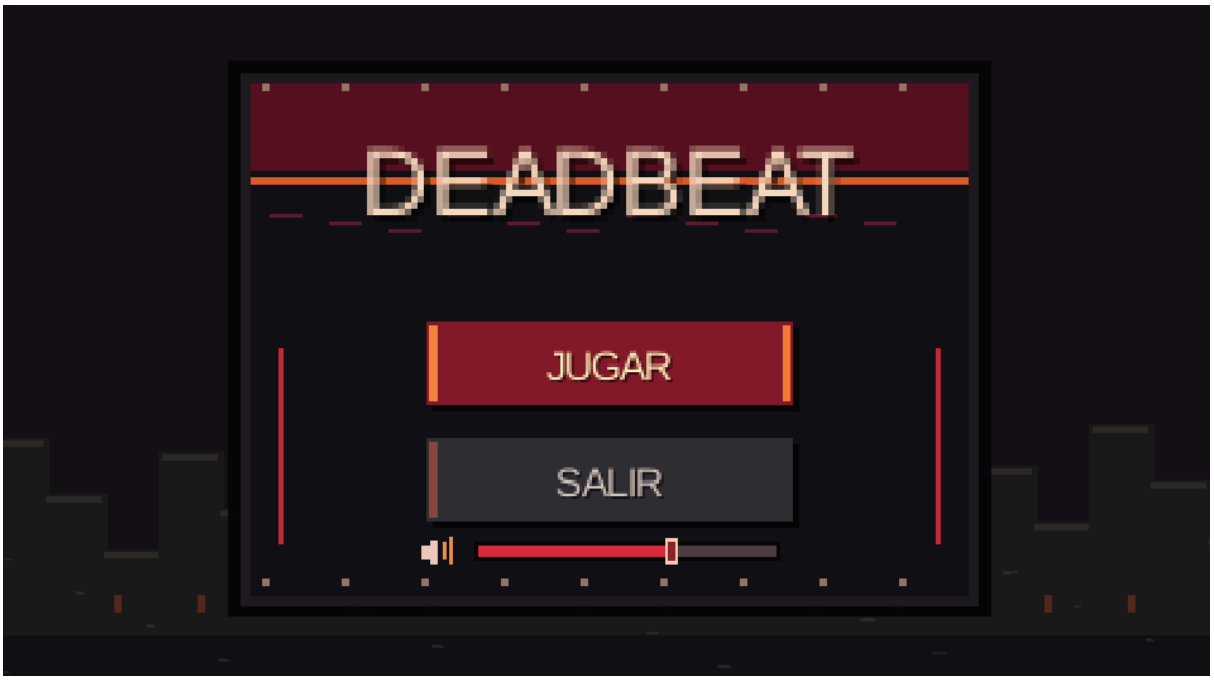
Proyecto Videojuego (GM) – DEADBEAT

Integrantes:
Sebastián Arriagada
Joaquín Carrillo
Benjamín Beltrán

GM-1: Explicación del juego desde la perspectiva del jugador

Requerimiento: Explicar el juego desde la perspectiva del jugador, incluyendo objetivos, narrativa, elementos y controles, utilizando maquetas o pantallazos.

DEADBEAT es un videojuego de acción y supervivencia en el que el jugador toma el papel de un músico que debe enfrentarse a hordas de zombies en una ciudad postapocalíptica. En un escenario donde la ciudad ha sido invadida por muertos vivientes, el protagonista utiliza sus instrumentos musicales como armas para sobrevivir. El juego combina una estética inspirada en la música alternativa con mecánicas de combate automático, progresión de personaje y supervivencia por oleadas.



Objetivo del juego

El objetivo principal es sobrevivir a cinco oleadas de zombies progresivamente más difíciles, evitando que los enemigos reduzcan la vida del protagonista a cero. Para conseguirlo, el jugador debe desplazarse constantemente por el escenario, esquivar enemigos y aprovechar las mejoras que obtiene durante la partida. La victoria se alcanza cuando finaliza la quinta oleada y se eliminan todos los zombies restantes. Por otro lado, si la vida del protagonista llega a cero, la partida termina en derrota.

Mecánicas y elementos principales

Movimiento y combate: El jugador puede desplazarse libremente por el escenario urbano. Los ataques se realizan automáticamente mediante dos instrumentos: una guitarra eléctrica que dispara proyectiles y unas baquetas que atacan a los enemigos cercanos.

Tipos de enemigos: Existen tres variedades de zombies: normales, rápidos y tanques, cada uno con características distintas de movimiento y resistencia.

Sistema de oleadas: Los enemigos aparecen de manera progresiva durante cinco oleadas, aumentando la dificultad a medida que avanza la partida.



Experiencia y niveles: Al derrotar zombies aparecen orbes de experiencia que el jugador puede recoger. La experiencia acumulada permite subir de nivel, y su progreso se representa mediante una barra ubicada en la parte inferior de la pantalla.

Sistema de mejoras: Cada vez que el personaje sube de nivel, puede seleccionar una mejora que fortalece sus capacidades. El juego dispone de seis tipos de mejoras, permitiendo potenciar al protagonista durante la partida.

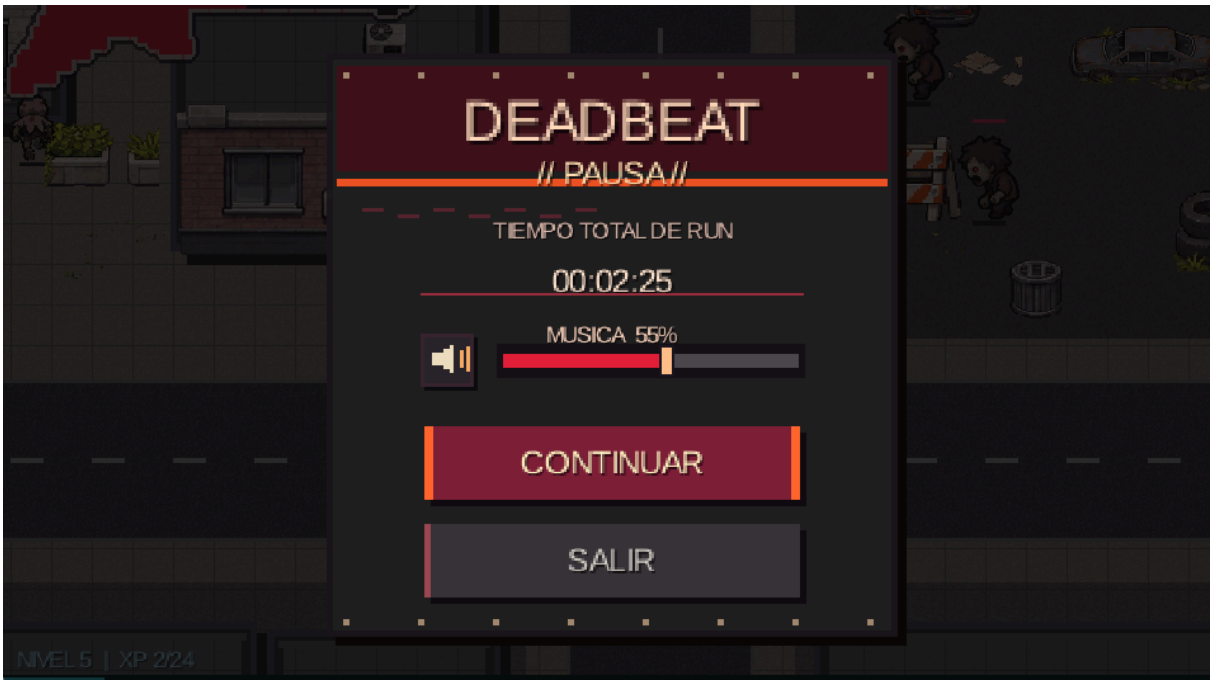


Controles

El personaje cuenta con movimiento en ocho direcciones mediante WASD o las flechas del teclado. En la pantalla principal se puede seleccionar JUGAR o SALIR mediante teclado o mouse. Durante la partida, ESC permite pausar o reanudar y R permite reiniciar la partida. Q y E permiten cambiar la pista musical.

Pantallas y estados del juego

El juego incorpora un menú principal, una pantalla de partida y paneles superpuestos para pausa y resultado. También incluye información visual de la vida del jugador y elementos de música. La experiencia y el nivel del jugador se administran en la clase Player; la generación y recolección de orbes se coordinan desde GameWorld.



El juego incluye también una pantalla de victoria, que aparece al completar las cinco oleadas y eliminar todos los enemigos restantes, y una pantalla de derrota cuando el protagonista pierde toda su vida.





Estas pantallas permiten comunicar claramente el estado de la partida y ofrecen al jugador una experiencia completa, desde el inicio hasta su finalización.

Evidencia en el código fuente

- MainMenuScreen.java: menú principal, opciones JUGAR/SALIR y controles de selección (aprox. líneas 21–29 y 98–162).
- GameScreen.java: controles de pausa, reinicio, actualización y resultado de la partida (aprox. líneas 137–195 y 205–216).
- Player.java: movimiento mediante WASD/flechas, vida y experiencia (aprox. líneas 106–127, 187–198 y 47–70).

- GameWorld.java: oleadas, aparición de enemigos, experiencia y condición de finalización (aprox. líneas 166–195 y 287–299).

GM-2: Análisis desde la perspectiva de ingeniería de software

El equipo identifica SpaceNav2024 y GameLluviaMenu2024 como referencias de diseño de DEADBEAT. El análisis de la implementación desarrollada se centra en la coordinación del movimiento, el combate automático, las colisiones, la progresión del jugador y las oleadas. GameScreen administra la presentación y la entrada del usuario, mientras que GameWorld coordina la simulación. Los ataques se organizan mediante Weapon y sus subclases, y el movimiento de los zombies mediante MovementStrategy.

La implementación de un videojuego de supervivencia requiere coordinar distintos sistemas que funcionan simultáneamente (como movimiento, combate, inteligencia de enemigos, colisiones, progresión del jugador y generación de oleadas), y concentrar todas estas responsabilidades en una sola clase dificultaría excesivamente la lectura, modificación y extensión del codigo. Es por esto que, se planteó una arquitectura que separa la presentación del juego, la lógica del mundo, las entidades y los comportamientos particulares.

Comparación de videojuegos de referencia.

Aspecto	Referencia conceptual	Implementación en DEADBEAT
Movimiento	Desplazamiento de entidades	Movimiento urbano con colisiones
Enemigos	Interacción entre entidades	Tres tipos de zombies
Combate	Proyectiles e impactos	Dos armas automáticas
Progresión	Actualización del estado	Experiencia, niveles y seis mejoras
Organización	Pantallas y objetos	GameScreen, GameWorld y entidades
Dificultad	Mecánicas de juego	Cinco oleadas progresivas

Para resolver estos problemas se incorporaron las siguientes decisiones de diseño:

- **Separación de responsabilidades:** GameScreen administra la presentación y la interacción del usuario, mientras que GameWorld coordina la simulación, las colisiones, las oleadas y las entidades.
- **Polimorfismo y abstracción:** La clase abstracta Weapon permite representar distintos elementos de combate mediante un contrato común, mientras que la interfaz Damageable permite aplicar daño a diferentes entidades.
- **Comportamientos intercambiables:** El patrón Strategy permite que Zombie delegue su desplazamiento a implementaciones de MovementStrategy, evitando concentrar todos los algoritmos de movimiento en la entidad.

- **Reutilización de algoritmos:** Template Method establece una secuencia común de actualización para las armas, mientras que cada implementación define su ataque específico.
- **Construcción controlada:** Builder permite configurar las oleadas mediante WaveBuilder, separando la definición de sus parámetros de la lógica que las utiliza.

Como resultado, DEADBEAT organiza sus funcionalidades en distintos componentes con responsabilidades claras y definidas, proponiendo así una organización adecuada del software y coordinando eficientemente los distintos sistemas que lo componen.

Evidencia en el código fuente

La implementación descrita se encuentra en GameScreen, que administra la presentación y la entrada del usuario; GameWorld, que coordina la simulación, las colisiones y las oleadas; Weapon y sus subclases, que implementan los ataques; MovementStrategy y sus implementaciones, que definen el movimiento de los zombies; y WaveBuilder, que construye las configuraciones de oleadas.

GM-3: Diseño UML de clases del dominio

Requerimiento: Diseñar un diagrama UML con clases del dominio del juego (clases base LibGDX y clases propias) y su implementación en Java.

La estructura del proyecto combina clases proporcionadas por LibGDX con clases propias que representan la lógica del videojuego. DeadbeatGame hereda de Game y actúa como punto de entrada de la aplicación. MainMenuScreen y GameScreen heredan de ScreenAdapter para implementar las pantallas. GameScreen mantiene una instancia de GameWorld, que coordina el escenario y las entidades de la partida.

Dentro del dominio, Player y Zombie implementan la interfaz Damageable, que define las operaciones comunes para recibir daño y consultar si una entidad continúa activa. Zombie mantiene una referencia a MovementStrategy, implementada por ChaseMovement y ZigzagMovement. Player mantiene una colección de objetos Weapon, cuya clase abstracta es extendida por ElectricGuitar y Drumsticks. Finalmente, WaveBuilder construye objetos Wave que contienen la configuración de cada oleada.

Evidencia en el código fuente

- DeadbeatGame.java: herencia desde com.badlogic.gdx.Game.
- MainMenuScreen.java y GameScreen.java: herencia desde ScreenAdapter.
- GameWorld.java: composición y coordinación de Player, Zombie, Projectile, ExperienceOrb y Wave.
- Player.java y Zombie.java: implementación de Damageable.
- MovementStrategy.java, ChaseMovement.java y ZigzagMovement.java: interfaz y estrategias concretas.
- Weapon.java, ElectricGuitar.java y Drumsticks.java: clase abstracta y especializaciones.

- Wave.java y WaveBuilder.java: configuración inmutable de oleadas y construcción.

Diagrama UML:

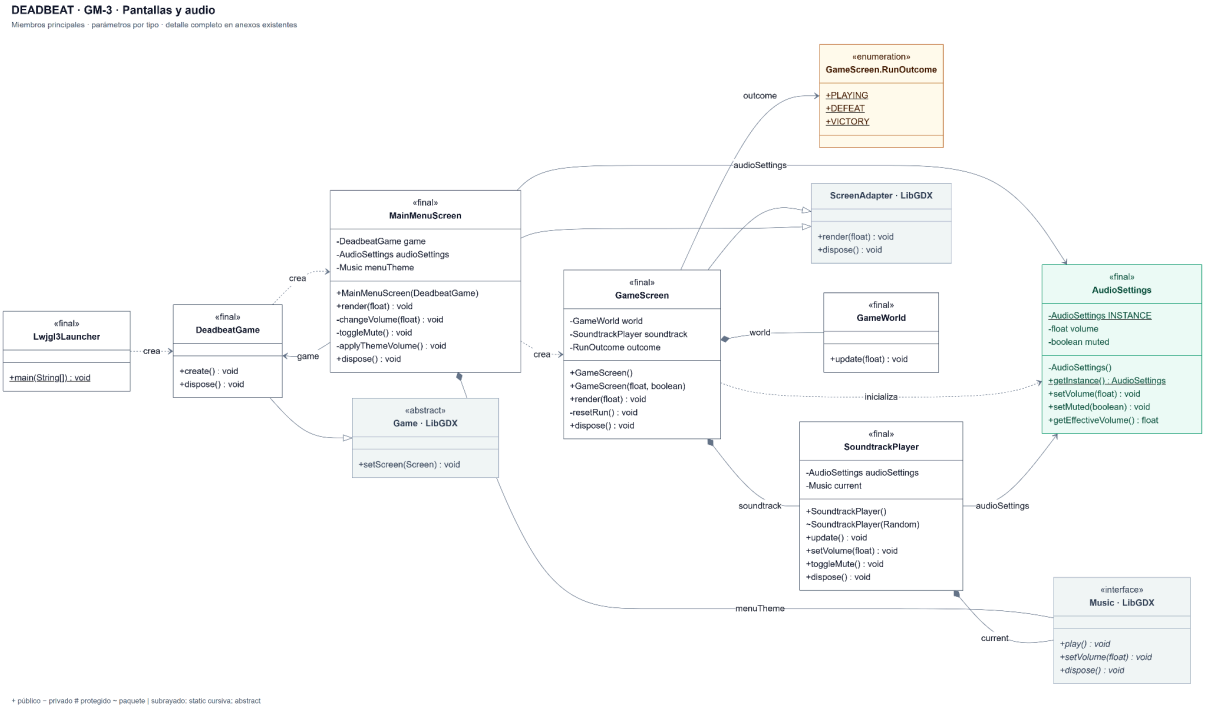


Figura 3.1. Diagrama UML del subsistema de pantallas y audio. Se presentan las relaciones entre el menú principal, la pantalla de juego, el reproductor musical y la configuración compartida de audio mediante el patrón Singleton. También se incluyen las principales clases e interfaces de LibGDX involucradas.

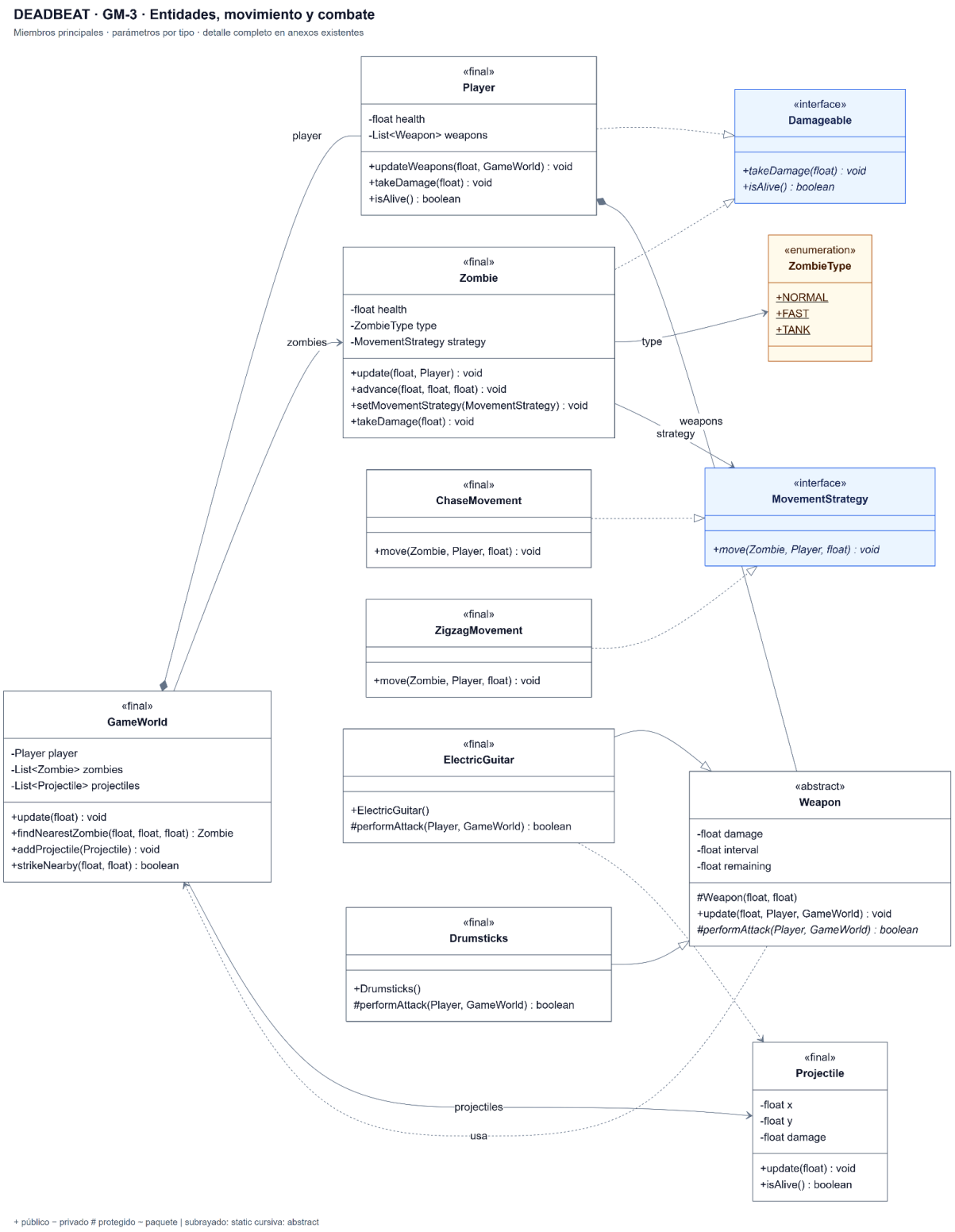


Figura 3.2. Diagrama UML del subsistema de entidades, movimiento y combate. *Se representan las entidades principales del videojuego, sus relaciones y la aplicación de abstracción, herencia, interfaces y polimorfismo en las armas y estrategias de movimiento.*

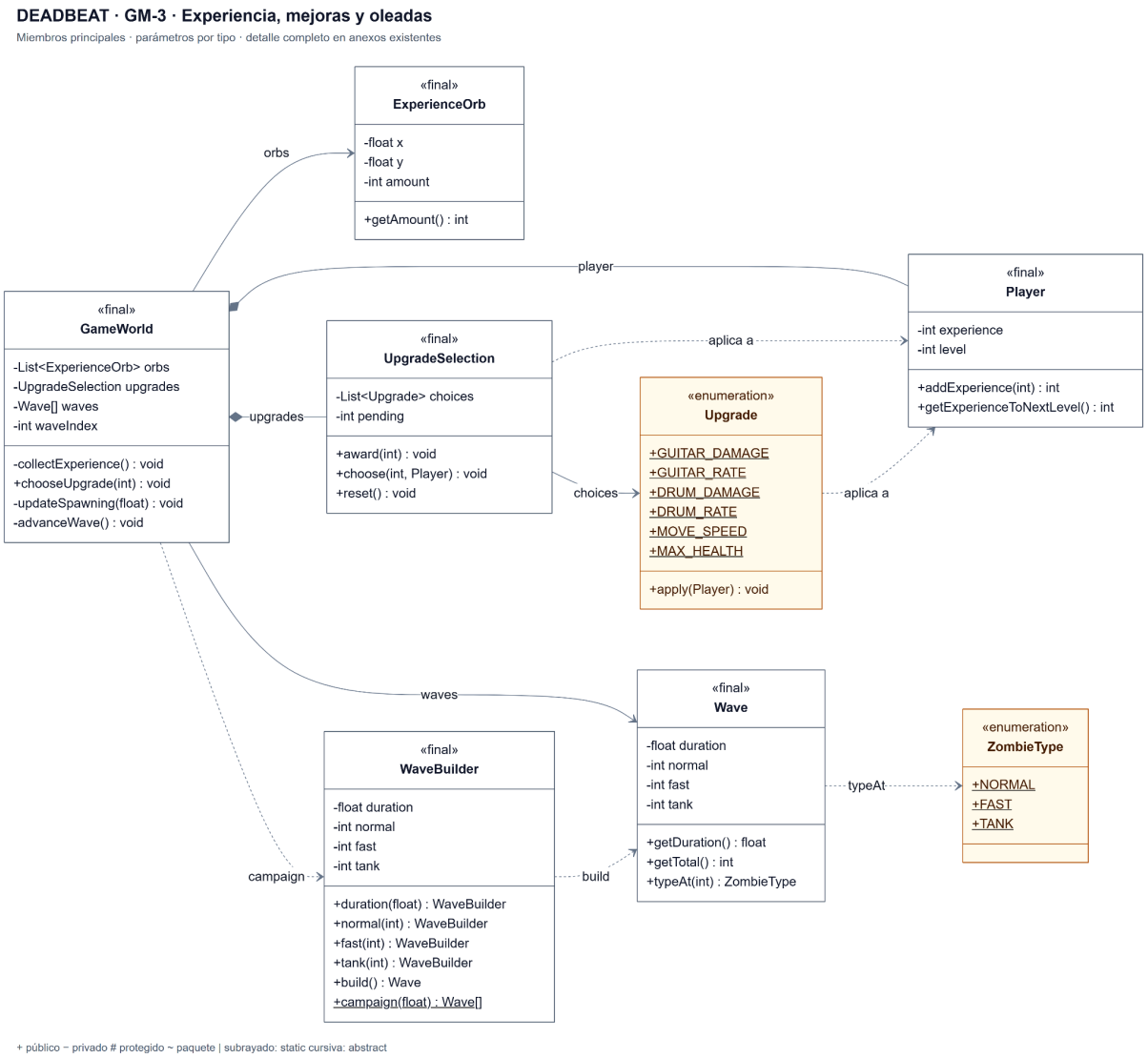


Figura 3.3. Diagrama UML del subsistema de experiencia, mejoras y oleadas. *Se muestra la organización del sistema de progresión, incluyendo la recolección de experiencia, selección de mejoras y construcción de oleadas mediante el patrón Builder.*

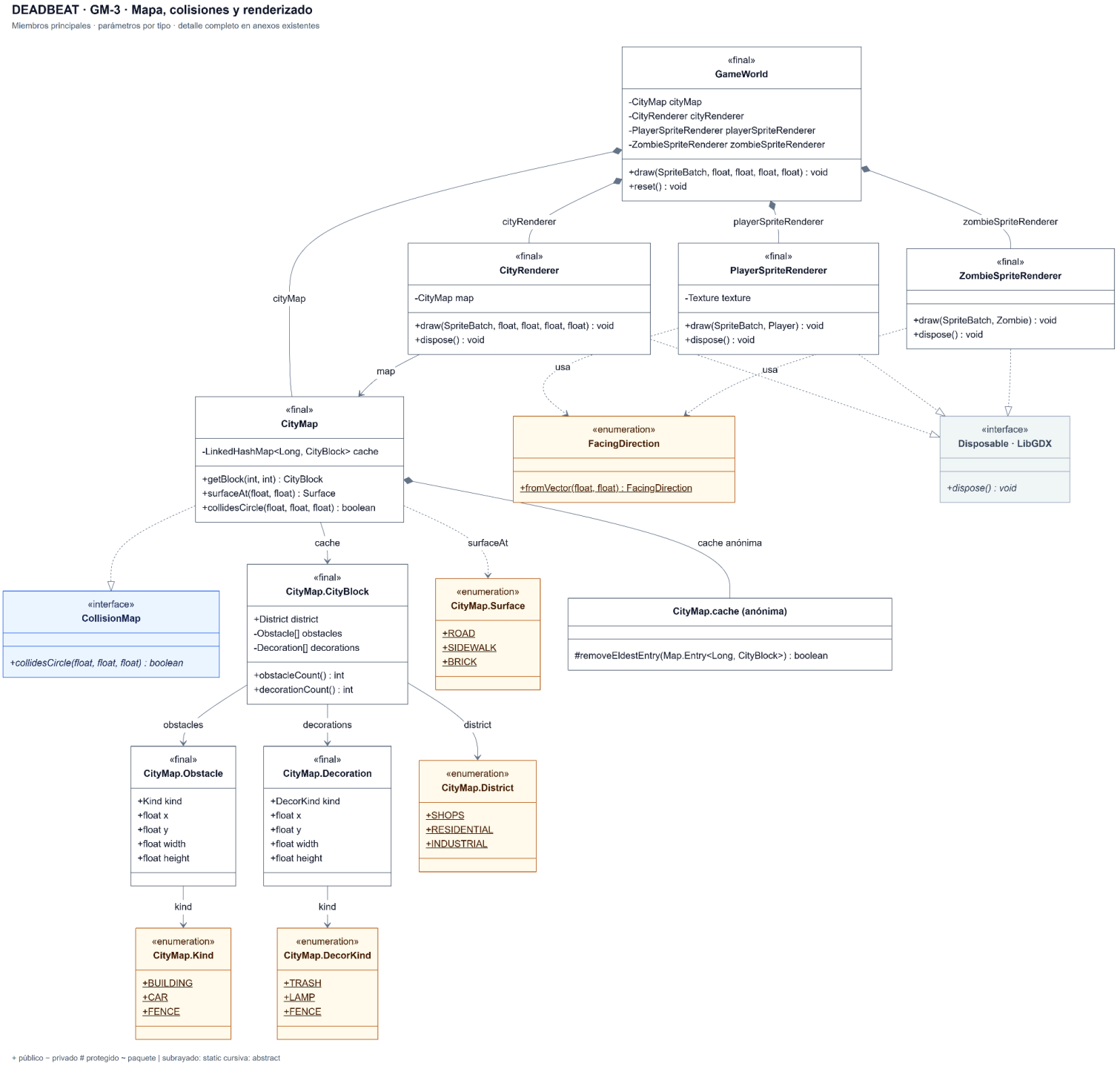


Figura 3.4. Diagrama UML del subsistema de mapa, colisiones y renderizado. *Se presentan las clases responsables de la generación y representación del escenario, la detección de colisiones y la renderización de entidades y elementos del mapa.*

Los diagramas anteriores presentan una vista organizada de la arquitectura del videojuego, destacando las clases y relaciones principales de cada subsistema. Para mantener su legibilidad, se muestran únicamente los miembros más relevantes. El detalle completo de atributos, constructores, métodos y relaciones se encuentra disponible en el **Anexo A: Documentación UML detallada del proyecto**.

GM-4: Clase abstracta e interfaz

Requerimiento: Diseñar y codificar una clase abstracta que sea padre de al menos dos clases y sea utilizada en el programa, además de una interfaz implementada por al menos dos clases y utilizada en el programa.

El proyecto utiliza la clase abstracta Weapon como base para representar el comportamiento compartido de los elementos de combate del jugador. Esta clase concentra atributos comunes, como el daño y el intervalo de actualización, y define el método abstracto performAttack(), que obliga a cada subclase a implementar su propia acción.

A partir de Weapon se implementan ElectricGuitar y Drumsticks. Ambas se incorporan al equipamiento del jugador desde Player.reset() y se actualizan mediante updateWeapons(), que recorre la colección utilizando el tipo base Weapon. De esta manera, la abstracción no queda solamente declarada, sino que participa en el funcionamiento del juego.

También se define la interfaz Damageable, que establece los métodos takeDamage() e isAlive(). Las clases Player y Zombie implementan esta interfaz, ya que ambas entidades pueden recibir daño y dejar de estar activas. GameWorld utiliza referencias de tipo Damageable en su método de aplicación de daño, lo que permite tratar a ambas clases mediante un contrato común.

Evidencia en el código fuente

- Weapon.java: declaración de la clase abstracta y método abstracto performAttack().
- ElectricGuitar.java y Drumsticks.java: clases que extienden Weapon.
- Player.java: implementa Damageable, mantiene la colección de Weapon y actualiza sus elementos.
- Damageable.java: declaración de la interfaz.
- Zombie.java: implementa Damageable.
- GameWorld.java: método damage(Damageable target, float amount), que utiliza la interfaz.

GM-5: Encapsulamiento y principios de orientación a objetos

Requerimiento: Aplicar encapsulamiento y principios de orientación a objetos en el diseño y la implementación del videojuego.

El proyecto aplica encapsulamiento mediante atributos privados en sus clases principales. En Player, por ejemplo, la posición, la velocidad, la vida, la experiencia, el nivel y la colección de elementos equipados se mantienen como estado interno. El acceso a estos datos se realiza mediante métodos públicos que exponen la información necesaria para el resto del sistema.

Zombie también mantiene privados sus atributos de estado, como el tipo, la posición, la vida y la estrategia de movimiento. En lugar de permitir que otras clases modifiquen directamente

su estado interno, ofrece métodos para actualizar su comportamiento, recibir daño y consultar sus características.

Se aplican además principios de orientación a objetos mediante abstracción, herencia, composición, interfaces y polimorfismo. Weapon concentra el comportamiento común de las implementaciones concretas; Damageable define un contrato compartido por Player y Zombie; y MovementStrategy permite delegar el comportamiento de movimiento en objetos especializados.

La distribución de responsabilidades también contribuye a la modularidad: GameScreen se encarga de la presentación y entrada de usuario, GameWorld coordina la simulación y las entidades administran su propio estado. En Player, getWeapons() entrega una vista no modificable de la colección, evitando que otros componentes alteren directamente el equipamiento.

Las colecciones expuestas mediante vistas no modificables impiden que componentes externos agreguen o eliminen elementos directamente. Sin embargo, los objetos contenidos pueden conservar estado mutable, por lo que la protección se aplica a la estructura de las colecciones y no implica necesariamente la inmutabilidad de sus elementos.

Evidencia en el código fuente

- Player.java: atributos privados (aprox. líneas 27–45), métodos de acceso y colección de armas no modificable (líneas 73–80 y 238–248).
- Zombie.java: atributos privados y acceso controlado a su estrategia (aprox. líneas 15–23 y 115–121).
- GameWorld.java: colecciones privadas de zombies, proyectiles y orbes; getters que entregan vistas no modificables.
- Weapon.java: abstracción del comportamiento común.
- Damageable.java y MovementStrategy.java: contratos mediante interfaces.

GM-6: Patrón de diseño Singleton

Problema

DEADBEAT utiliza música tanto en el menú principal como durante la partida. Al administrar el volumen y el silenciamiento desde componentes independientes, podían producirse inconsistencias entre las preferencias de audio de ambas pantallas. Era necesario disponer de una configuración compartida que permitiera modificar el volumen y el estado de silencio desde distintos componentes sin mantener copias independientes de esos valores.

Contexto

El videojuego utiliza distintas pantallas y reproductores de audio durante su ejecución. Aunque cada reproductor administra sus propias pistas musicales, todos deben respetar las mismas preferencias del usuario. La configuración debe conservarse al cambiar de pantalla o reiniciar una partida, sin depender del ciclo de vida de cada reproductor.

Solución

Se implementó el patrón Singleton mediante la clase AudioSettings, ubicada en el paquete com.deadbeat.audio. La clase es final, posee un constructor privado y declara una única instancia estática y final (INSTANCE), accesible mediante getInstance(). La instancia administra dos atributos privados: volume, que almacena el volumen seleccionado, y muted, que indica si el audio está silenciado. Los métodos setVolume(), setMuted() y toggleMute() permiten modificar las preferencias, mientras que getEffectiveVolume() determina el volumen efectivo considerando el estado de silencio. El método setVolume() limita los valores al intervalo entre 0 y 1 e ignora valores no finitos. MainMenuScreen y SoundtrackPlayer utilizan la configuración compartida. GameScreen accede a ella mediante su reproductor, permitiendo que menú y partida respeten las mismas preferencias de audio. De esta manera, los reproductores conservan la responsabilidad de reproducir música, mientras que AudioSettings centraliza exclusivamente la configuración global.

Justificación del patrón

Singleton resulta apropiado porque DEADBEAT necesita una sola configuración de audio compartida durante la ejecución de la aplicación. La instancia no se recrea al cambiar de pantalla ni al reiniciar una partida, por lo que las preferencias se mantienen independientes del ciclo de vida de los componentes gráficos y musicales. Esto evita estados de volumen contradictorios y proporciona un punto único de acceso a las preferencias.

Evidencia en el código

- AudioSettings.java: implementación del patrón Singleton, atributos privados y operaciones de configuración.
- MainMenuScreen.java: consulta y actualización de preferencias compartidas.
- SoundtrackPlayer.java: aplicación del volumen efectivo a las pistas musicales.
- GameScreen.java: inicialización y utilización del reproductor con la configuración correspondiente.

Diagrama UML:

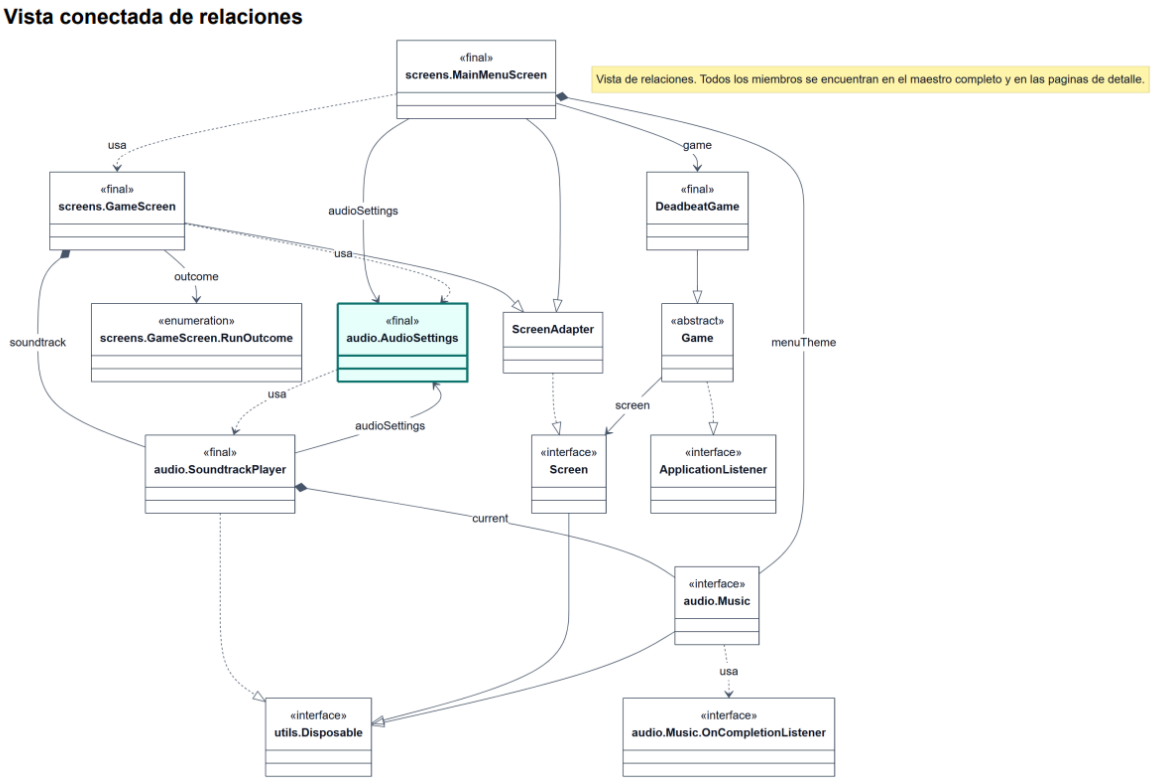


Figura 6.1. Diagrama UML de relaciones del patrón Singleton. *Se muestran las dependencias entre `AudioSettings` y los componentes encargados de administrar y reproducir la música del videojuego. La configuración compartida permite mantener preferencias de volumen y silenciamiento coherentes entre el menú principal y la partida.*

audio.AudioSettings <<class>>

DeadBeat/core/src/main/java/com/deadbeat/audio/AudioSettings.java:4
public final

ATRIBUTOS

```
-INSTANCE: AudioSettings {static, final}
-volume: float
-muted: boolean
```

CONSTRUCTORES EXPLICITOS

```
-AudioSettings()
```

METODOS DECLARADOS

```
+getInstance(): AudioSettings {static}
+getVolume(): float
+setVolume(float volume): void
+isMuted(): boolean
+setMuted(boolean muted): void
+toggleMute(): void
+getEffectiveVolume(): float
```

Singleton: constructor privado, INSTANCE static final y getInstance static. Las preferencias pertenecen al objeto.

Figura 6.2. Estructura UML de la clase AudioSettings. *Se presenta la implementación del patrón Singleton mediante un constructor privado, una instancia estática y final (`INSTANCE`) y el método público y estático `getInstance()`. La clase encapsula el volumen y el estado de silencio, proporcionando operaciones para consultar y modificar ambas preferencias.*

La clase AudioSettings centraliza las preferencias de audio durante la ejecución del videojuego. Las clases colaboradoras utilizan esta instancia compartida sin crear configuraciones independientes. El detalle de sus atributos, constructores, métodos y relaciones se presenta en el **Anexo B: Documentación UML detallada del patrón Singleton.**

GM-7: Patrón de diseño Strategy

Requerimiento: Aplicar el patrón Strategy, describiendo el problema, contexto, solución y UML.

El patrón Strategy se utiliza para permitir que los zombies tengan distintos comportamientos de desplazamiento sin concentrar todos los algoritmos en una única clase. El problema consiste en que los distintos tipos de enemigos necesitan moverse de manera diferente y, además, el comportamiento de algunos puede cambiar durante la partida.

Para resolverlo, se define la interfaz MovementStrategy, que establece el método move(). Las clases ChaseMovement y ZigzagMovement implementan ese contrato con algoritmos distintos. Zombie mantiene una referencia a MovementStrategy y delega en ella el desplazamiento.

La estrategia se asigna al crear el zombie según su tipo. Los zombies normales y tanque utilizan persecución directa, mientras que los rápidos utilizan zigzag. Además, Zombie puede alternar la estrategia del enemigo rápido durante la ejecución, cambiando entre ZigzagMovement y ChaseMovement sin reemplazar la clase Zombie.

Esta solución evita que la lógica de movimiento dependa de una gran cantidad de condicionales dentro de una sola clase y facilita incorporar nuevas estrategias sin modificar la estructura principal del enemigo.

Evidencia en el código fuente

- MovementStrategy.java: interfaz con el método move().
- ChaseMovement.java: implementación de persecución directa.
- ZigzagMovement.java: implementación de desplazamiento zigzagueante.
- Zombie.java: atributo strategy, asignación inicial y cambio de estrategia durante update() (aprox. líneas 23 y 41–50; métodos setMovementStrategy() y getMovementStrategy()).

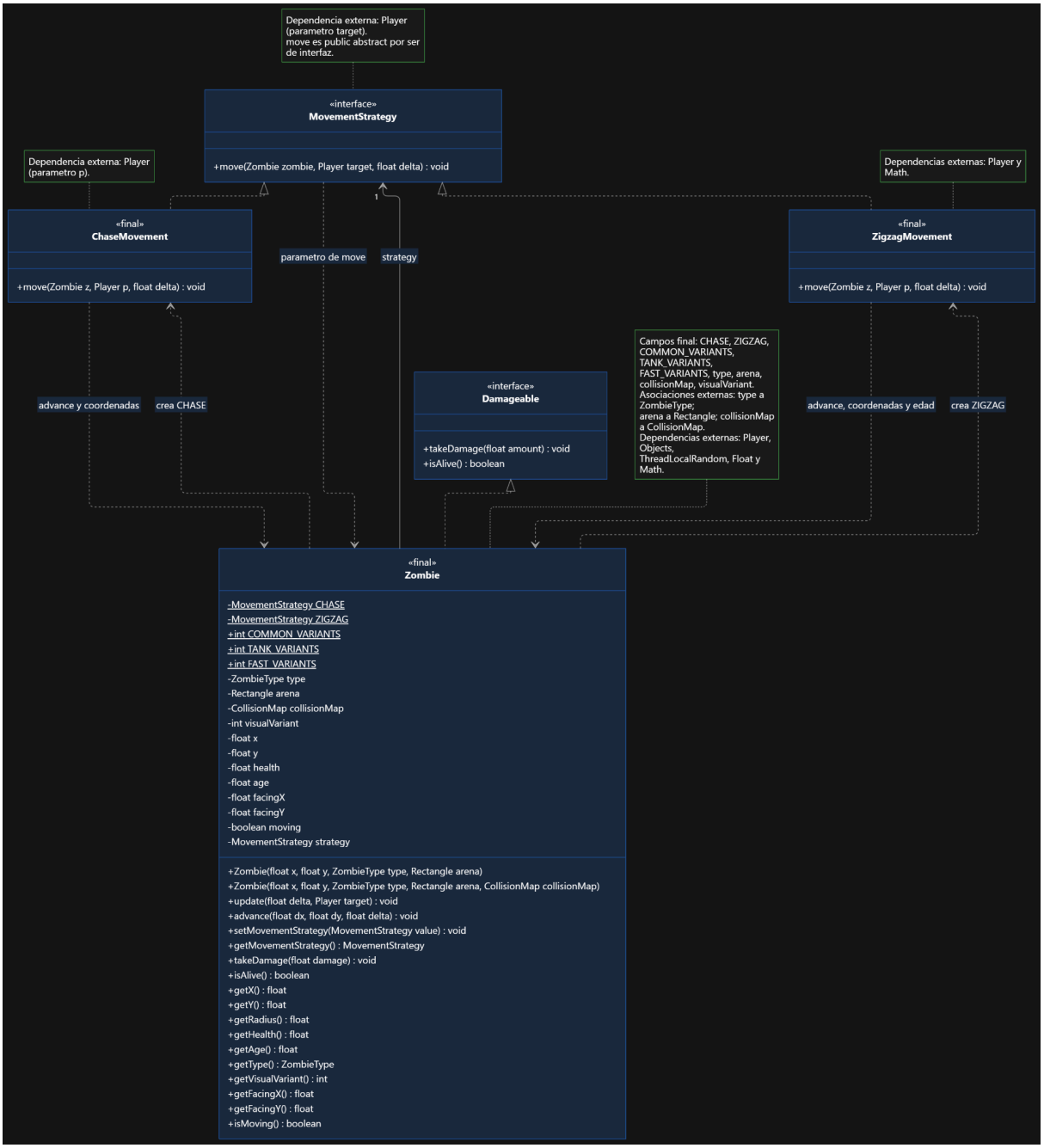


Figura 7.1. Diagrama UML del patrón Strategy. *Se representa la interfaz `MovementStrategy` y sus implementaciones `ChaseMovement` y `ZigzagMovement`, utilizadas por la clase `Zombie` para delegar el comportamiento de movimiento y permitir distintas estrategias durante la ejecución.*

GM-8: Patrón de diseño Template Method

Requerimiento: Aplicar el patrón Template Method, describiendo el problema, contexto, solución y UML.

El patrón Template Method se utiliza para definir una secuencia común de actualización para los elementos de combate del jugador, manteniendo la posibilidad de que cada implementación ejecute una acción distinta. El problema aparece porque los distintos elementos comparten la administración del tiempo de espera entre acciones, pero no realizan la misma operación.

La clase abstracta Weapon resuelve este problema mediante el método final update(). Este método descuenta el tiempo restante, comprueba si el elemento puede ejecutar su acción y luego invoca performAttack(). Si la acción se realiza correctamente, reinicia el intervalo de espera. La secuencia general queda definida en la clase padre.

Las subclases ElectricGuitar y Drumsticks implementan performAttack() de manera diferente. La primera busca un zombie cercano y genera un proyectil, mientras que la segunda delega un impacto de alcance cercano en GameWorld. De esta forma, ambas reutilizan el mismo algoritmo general de actualización sin duplicar la administración del intervalo.

El uso de update() como método final evita que las subclases alteren el orden de los pasos comunes y mantiene la variación limitada al método abstracto performAttack().

Evidencia en el código fuente

- Weapon.java: método final update() y método abstracto performAttack().
- ElectricGuitar.java: implementación concreta de performAttack().
- Drumsticks.java: implementación concreta de performAttack().
- Player.java: updateWeapons() recorre el equipamiento usando el tipo base Weapon.

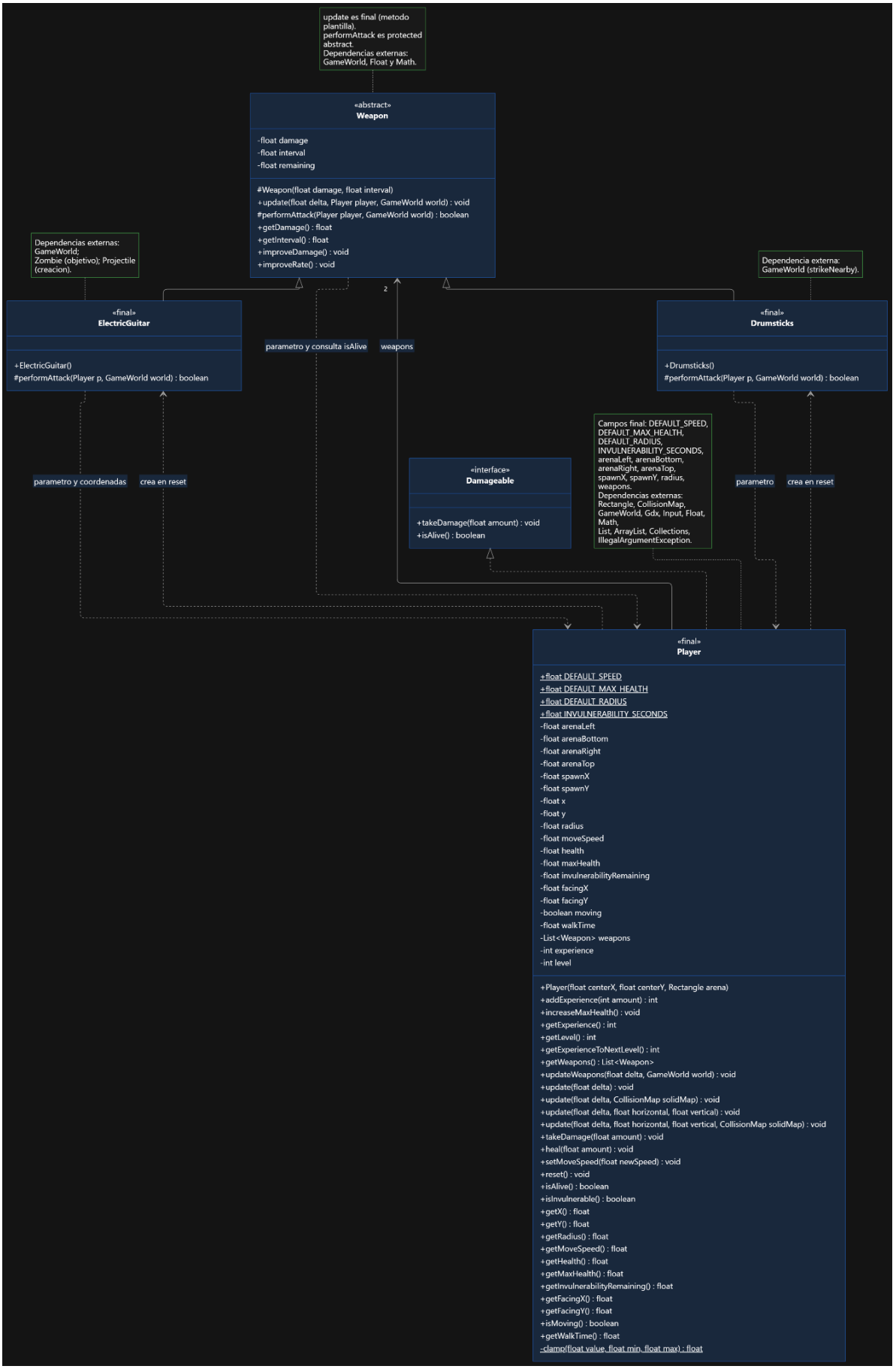

update es final (metodo plantilla), performAttack es protected abstract.
Dependencias externas: GameWorld, Float y Math.
«abstract»
Weapon
-float damage
-float interval
-float remaining
#Weapon(float damage, float interval)
+update(float delta, Player player, GameWorld world) : void
#performAttack(Player player, GameWorld world) : boolean
+getDamage() : float
+getInterval() : float
+improveDamage() : void
+improveRate() : void
Dependencias externas: GameWorld; Zombie (objetivo); Projectile (creacion).
Dependencia externa: GameWorld (strikeNearby).
2
«final»
ElectricGuitar
+ElectricGuitar()
#performAttack(Player p, GameWorld world) : boolean
parametro y consulta isAlive
weapons
«final»
Drumsticks
+Drumsticks()
#performAttack(Player p, GameWorld world) : boolean
Campos final: DEFAULT_SPEED, DEFAULT_MAX_HEALTH, DEFAULT_RADIUS, INVULNERABILITY_SECONDS, arenaLeft, arenaBottom, arenaRight, arenaTop, spawnX, spawnY, radius, weapons.
Dependencias externas: Rectangle, CollisionMap, GameWorld, Gdx, Input, Float, Math, List, ArrayList, Collections, IllegalArgumentException.
parametro y coordenadas
crea en reset
«interface»
Damageable
+takeDamage(float amount) : void
+isAlive() : boolean
parametro
crea en reset
«final»
Player
+float DEFAULT_SPEED
+float DEFAULT_MAX_HEALTH
+float DEFAULT_RADIUS
+float INVULNERABILITY_SECONDS
-float arenaLeft
-float arenaBottom
-float arenaRight
-float arenaTop
-float spawnX
-float spawnY
-float x
-float y
-float radius
-float moveSpeed
-float health
-float maxHealth
-float invulnerabilityRemaining
-float facingX
-float facingY
-boolean moving
-float walkTime
-List<Weapon> weapons
-int experience
-int level
+Player(float centerX, float centerY, Rectangle arena)
+addExperience(int amount) : int
+increaseMaxHealth() : void
+getExperience() : int
+getLevel() : int
+getExperienceToNextLevel() : int
+getWeapons() : List<Weapon>
+updateWeapons(float delta, GameWorld world) : void
+update(float delta) : void
+update(float delta, CollisionMap solidMap) : void
+update(float delta, float horizontal, float vertical) : void
+update(float delta, float horizontal, float vertical, CollisionMap solidMap) : void
+takeDamage(float amount) : void
+heal(float amount) : void
+setMoveSpeed(float newSpeed) : void
+reset() : void
+isAlive() : boolean
+isInvulnerable() : boolean
+getX() : float
+getY() : float
+getRadius() : float
+getMoveSpeed() : float
+getHealth() : float
+getMaxHealth() : float
+getInvulnerabilityRemaining() : float
+getFacingX() : float
+getFacingY() : float
+isMoving() : boolean
+getWalkTime() : float
-clamp(float value, float min, float max) : float

***Figura 8.1. Diagrama UML del patrón Template Method**. Se presenta la jerarquía de armas encabezada por la clase abstracta `Weapon`, cuyo método `update()` define la estructura general del ataque. Las subclases `ElectricGuitar` y `Drumsticks` implementan el comportamiento específico mediante `performAttack()`.*

GM-9: Patrón de diseño Builder

Requerimiento: Aplicar el patrón Builder, describiendo el problema, contexto, solución y UML.

El patrón Builder se utiliza para construir las configuraciones de las oleadas del videojuego. Cada oleada requiere definir una duración y cantidades de zombies normales, rápidos y tanque. Construir estas configuraciones mediante una secuencia de parámetros independientes puede dificultar su lectura y validación.

Para resolverlo, se implementó WaveBuilder con métodos encadenables duration(), normal(), fast() y tank(). El método build() valida que la duración sea positiva, que las cantidades no sean negativas y que la configuración contenga una cantidad válida de enemigos. Si los parámetros no cumplen estas condiciones, se lanza IllegalArgumentException.

El resultado de build() es un objeto Wave, que mantiene sus atributos como valores finales. Esto permite que la configuración construida no se modifique accidentalmente después de su creación.

El método estático campaign() construye cinco configuraciones de oleada utilizando WaveBuilder. GameWorld obtiene estas configuraciones al inicializarse y las utiliza para administrar la aparición progresiva de enemigos. Así, Builder participa en la ejecución real del videojuego y no queda como una clase aislada.

Evidencia en el código fuente

- WaveBuilder.java: métodos encadenables, validación en build() y método campaign() (aprox. líneas 2–43).
- Wave.java: objeto de configuración con atributos finales y métodos de consulta.
- GameWorld.java: invocación de WaveBuilder.campaign() al construir el mundo (aprox. líneas 42–53) y administración del avance de oleadas (líneas 166–195).

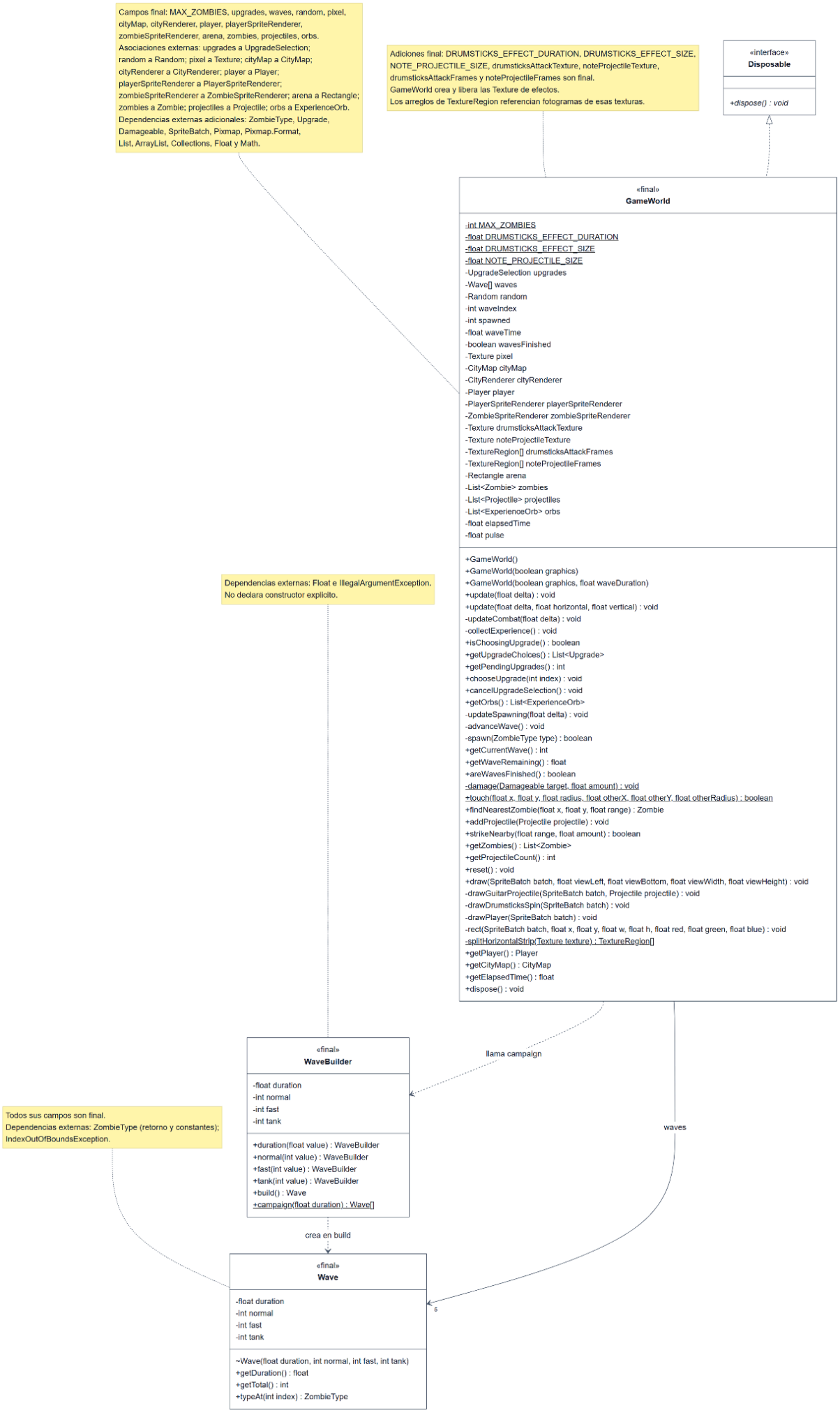

Campos final: MAX_ZOMBIES, upgrades, waves, random, pixel, cityMap, cityRenderer, player, playerSpriteRenderer, zombieSpriteRenderer, arena, zombies, projectiles, orbs.
Asociaciones externas: upgrades a UpgradeSelection; random a Random; pixel a Texture; cityMap a CityMap; cityRenderer a CityRenderer; player a Player; playerSpriteRenderer a PlayerSpriteRenderer; zombieSpriteRenderer a ZombieSpriteRenderer; arena a Rectangle; zombies a Zombie; projectiles a Projectile; orbs a ExperienceOrb.
Dependencias externas adicionales: ZombieType, Upgrade, Damageable, SpriteBatch, Pixmap, Pixmap.Format, List, ArrayList, Collections, Float y Math.
Adiciones final: DRUMSTICKS_EFFECT_DURATION, DRUMSTICKS_EFFECT_SIZE, NOTE_PROJECTILE_SIZE, drumsticksAttackTexture, noteProjectileTexture, drumsticksAttackFrames y noteProjectileFrames son final.
GameWorld crea y libera las Texture de efectos.
Los arreglos de TextureRegion referencian fotogramas de esas texturas.
«interface»
Disposable
+dispose() : void
«final»
GameWorld
-int MAX_ZOMBIES
-float DRUMSTICKS_EFFECT_DURATION
-float DRUMSTICKS_EFFECT_SIZE
-float NOTE_PROJECTILE_SIZE
-UpgradeSelection upgrades
-Wave[] waves
-Random random
-int waveIndex
-int spawned
-float waveTime
-boolean wavesFinished
-Texture pixel
-CityMap cityMap
-CityRenderer cityRenderer
-Player player
-PlayerSpriteRenderer playerSpriteRenderer
-ZombieSpriteRenderer zombieSpriteRenderer
-Texture drumsticksAttackTexture
-Texture noteProjectileTexture
-TextureRegion[] drumsticksAttackFrames
-TextureRegion[] noteProjectileFrames
-Rectangle arena
-List<Zombie> zombies
-List<Projectile> projectiles
-List<ExperienceOrb> orbs
-float elapsedTime
-float pulse
+GameWorld()
+GameWorld(boolean graphics)
+GameWorld(boolean graphics, float waveDuration)
+update(float delta) : void
+update(float delta, float horizontal, float vertical) : void
-updateCombat(float delta) : void
-collectExperience() : void
+isChoosingUpgrade() : boolean
+getUpgradeChoices() : List<Upgrade>
+getPendingUpgrades() : int
+chooseUpgrade(int index) : void
+cancelUpgradeSelection() : void
+getOrbs() : List<ExperienceOrb>
-updateSpawning(float delta) : void
-advanceWave() : void
-spawn(ZombieType type) : boolean
+getCurrentWave() : int
+getWaveRemaining() : float
+areWavesFinished() : boolean
-damage(Damageable target, float amount) : void
+touch(float x, float y, float radius, float otherX, float otherY, float otherRadius) : boolean
+findNearestZombie(float x, float y, float range) : Zombie
+addProjectile(Projectile projectile) : void
+strikeNearby(float range, float amount) : boolean
+getZombies() : List<Zombie>
+getProjectileCount() : int
+reset() : void
+draw(SpriteBatch batch, float viewLeft, float viewBottom, float viewWidth, float viewHeight) : void
-drawGuitarProjectile(SpriteBatch batch, Projectile projectile) : void
-drawDrumsticksSpin(SpriteBatch batch) : void
-drawPlayer(SpriteBatch batch) : void
-rect(SpriteBatch batch, float x, float y, float w, float h, float red, float green, float blue) : void
-splitHorizontalStrip(Texture texture) : TextureRegion[]
+getPlayer() : Player
+getCityMap() : CityMap
+getElapsedTime() : float
+dispose() : void
Dependencias externas: Float e IllegalArgumentException.
No declara constructor explícito.
«final»
WaveBuilder
-float duration
-int normal
-int fast
-int tank
+duration(float value) : WaveBuilder
+normal(int value) : WaveBuilder
+fast(int value) : WaveBuilder
+tank(int value) : WaveBuilder
+build() : Wave
+campaign(float duration) : Wave[]
llama campaign
Todos sus campos son final.
Dependencias externas: ZombieType (retorno y constantes); IndexOutOfBoundsException.
crea en build
waves
«final»
Wave
-float duration
-int normal
-int fast
-int tank
~Wave(float duration, int normal, int fast, int tank)
+getDuration() : float
+getTotal() : int
+typeAt(int index) : ZombieType
6

Figura 9.1. Diagrama UML del patrón Builder. Se muestra cómo `WaveBuilder` construye objetos `Wave` mediante métodos de configuración encadenados. Las oleadas generadas son utilizadas por `GameWorld` para organizar la progresión de la partida.

GM-10: Utilización de GitHub

Requerimiento: Utilizar GitHub y realizar al menos seis commits durante el desarrollo del proyecto.

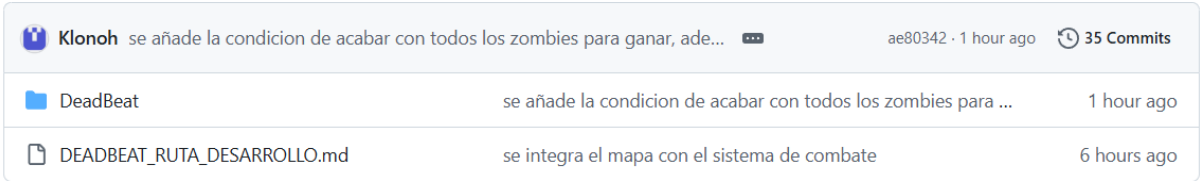
El proyecto se encuentra alojado en un repositorio de GitHub, donde se mantiene el código fuente y la documentación asociada a su desarrollo. El historial del repositorio permite observar la evolución del videojuego y los cambios realizados durante su implementación.

El desarrollo de DEADBEAT se encuentra versionado mediante Git y alojado en GitHub. El historial del repositorio supera el mínimo de seis commits exigido por la pauta y permite revisar la evolución del proyecto. El historial permite revisar los cambios incorporados en el proyecto y constituye evidencia del uso de control de versiones.

Evidencia en el código fuente

Repositorio: https://github.com/Klonoh/Deadbeat

Historial de commits: https://github.com/Klonoh/Deadbeat/commits/main



Anexos

Anexo A. Documentación UML detallada del proyecto (GM-3).

Contiene el inventario de clases y sus relaciones, además del detalle de atributos, constructores y métodos de las clases del videojuego. Complementa las cuatro vistas UML por subsistemas incluidas en el informe principal.

Archivo adjunto: *Anexo_A_GM3_UML_Completo.pdf*.

Anexo B. Documentación UML del patrón Singleton (GM-6).

Contiene el modelo UML completo de la implementación de Singleton mediante `AudioSettings`, sus clases colaboradoras y la trazabilidad entre el código Java y los elementos representados.

Archivo adjunto: *Anexo_B_GM6_Singleton.pdf.*